Visualización de datos en R

Mario Morales

Table of contents I

1. Transformación de datos
2. Datos estructurados (`tidy data`)
3. Alargamiento de datos: (`pivot_longer`)
4. Ampliación de datos: `pivot_wider()`

Section 1

Transformación de datos

Todos los conjuntos de datos ordenados son iguales, pero cada conjunto de datos desordenado es desordenado a su manera.

Introducción

- Aprenderá una forma consistente de organizar datos en R utilizando un sistema llamado `tidy data`.
- Poner los datos en este formato requiere algo de trabajo al principio, pero ese trabajo dará sus frutos a largo plazo.
- Una vez que tenga datos organizados y las herramientas proporcionadas por los paquetes en `tidyverse`, dedicará mucho menos tiempo pasando los datos de una representación a otra, lo que le permitirá dedicar más tiempo a las preguntas que le interesan sobre los datos.
- Aprenderemos la definición de `tidy data` y la aplicaremos a un conjunto de datos de *juguete* simple.

Introducción (cont)

- Estudiaremos la herramienta principal que utilizarás para organizar los datos: `pivoting`.
- El `pivoting` permite cambiar la forma de los datos sin cambiar ninguno de los valores.

Section 2

Datos estructurados (tidy data)

Tidy Data

- Se pueden representar los mismos datos de varias maneras.
- El siguiente ejemplo muestra los mismos datos organizados de tres maneras diferentes.
- Cada conjunto de datos muestra los mismos valores de cuatro variables: país (`country`), año (`year`), población (`population`) y número de casos (`cases`), documentados de tuberculosis (TB),
- Pero cada conjunto de datos organiza (estructura) los valores de manera diferente.

Tidy Data (tabla 1)

```r
library(tidyverse)
table1
```

```
# A tibble: 6 x 4
  country      year  cases population
  <chr>       <dbl>  <dbl>      <dbl>
1 Afghanistan  1999    745   19987071
2 Afghanistan  2000   2666   20595360
3 Brazil       1999  37737  172006362
4 Brazil       2000  80488  174504898
5 China        1999 212258 1272915272
6 China        2000 213766 1280428583
```

Tidy Data (tabla 2)

```
table2
```

```
# A tibble: 12 x 4
   country      year type            count
   <chr>       <dbl> <chr>           <dbl>
 1 Afghanistan  1999 cases             745
 2 Afghanistan  1999 population   19987071
 3 Afghanistan  2000 cases            2666
 4 Afghanistan  2000 population   20595360
 5 Brazil       1999 cases           37737
 6 Brazil       1999 population  172006362
 7 Brazil       2000 cases           80488
 8 Brazil       2000 population  174504898
 9 China        1999 cases          212258
10 China        1999 population 1272915272
11 China        2000 cases          213766
12 China        2000 population 1280428583
```

Tidy Data (tabla 3)

```
table3
```

```
# A tibble: 6 x 3
  country      year rate
  <chr>       <dbl> <chr>
1 Afghanistan  1999 745/19987071
2 Afghanistan  2000 2666/20595360
3 Brazil       1999 37737/172006362
4 Brazil       2000 80488/174504898
5 China        1999 212258/1272915272
6 China        2000 213766/1280428583
```

- Todas ellas son representaciones de los mismos datos, pero no son igualmente fáciles de usar. Una de ellas, la tabla 1, será mucho más fácil de usar dentro de `tidyverse` porque es `tidy` (organizada).

Reglas para que un data sea tidy (organizado, estructurado)

Hay tres reglas interrelacionadas que hacen que un conjunto de datos esté organizado (estructurado):

1. Cada variable es una columna; Cada columna es una variable.
2. Cada observación es una fila; Cada fila es una observación.
3. Cada valor es una celda; Cada celda es un valor único.

Ventajas de que los datos sean tidy

- ¿Por qué asegurarse de que sus datos estén organizados? Hay dos ventajas principales:
  - Existe una ventaja general en elegir una forma consistente de almacenar datos. Si tienes una estructura de datos consistente, es más fácil aprender las herramientas que trabajan con ella porque tienen una uniformidad subyacente.
  - Hay una ventaja específica en colocar variables en columnas porque permite que la naturaleza vectorizada de R brille. Como vimos con `mutate()` y `summarize()`, la mayoría de las funciones integradas de R trabajan con vectores de valores. Esto hace que la transformación de datos organizados resulte especialmente natural.
- `dplyr`, `ggplot2` y todos los demás paquetes de `tidyverse` están diseñados para trabajar con `tidy data` (datos estucturados).
- A continuación se muestran algunos ejemplos de cómo trabajar con la tabla 1:

Tasa por cada 10k habitantes

```r
table1 |> mutate(rate = cases / population * 10000)
```

```
# A tibble: 6 x 5
  country      year  cases population  rate
  <chr>       <dbl>  <dbl>      <dbl> <dbl>
1 Afghanistan  1999    745   19987071 0.373
2 Afghanistan  2000   2666   20595360 1.29
3 Brazil       1999  37737  172006362 2.19
4 Brazil       2000  80488  174504898 4.61
5 China        1999 212258 1272915272 1.67
6 China        2000 213766 1280428583 1.67
```

Total de casos por año

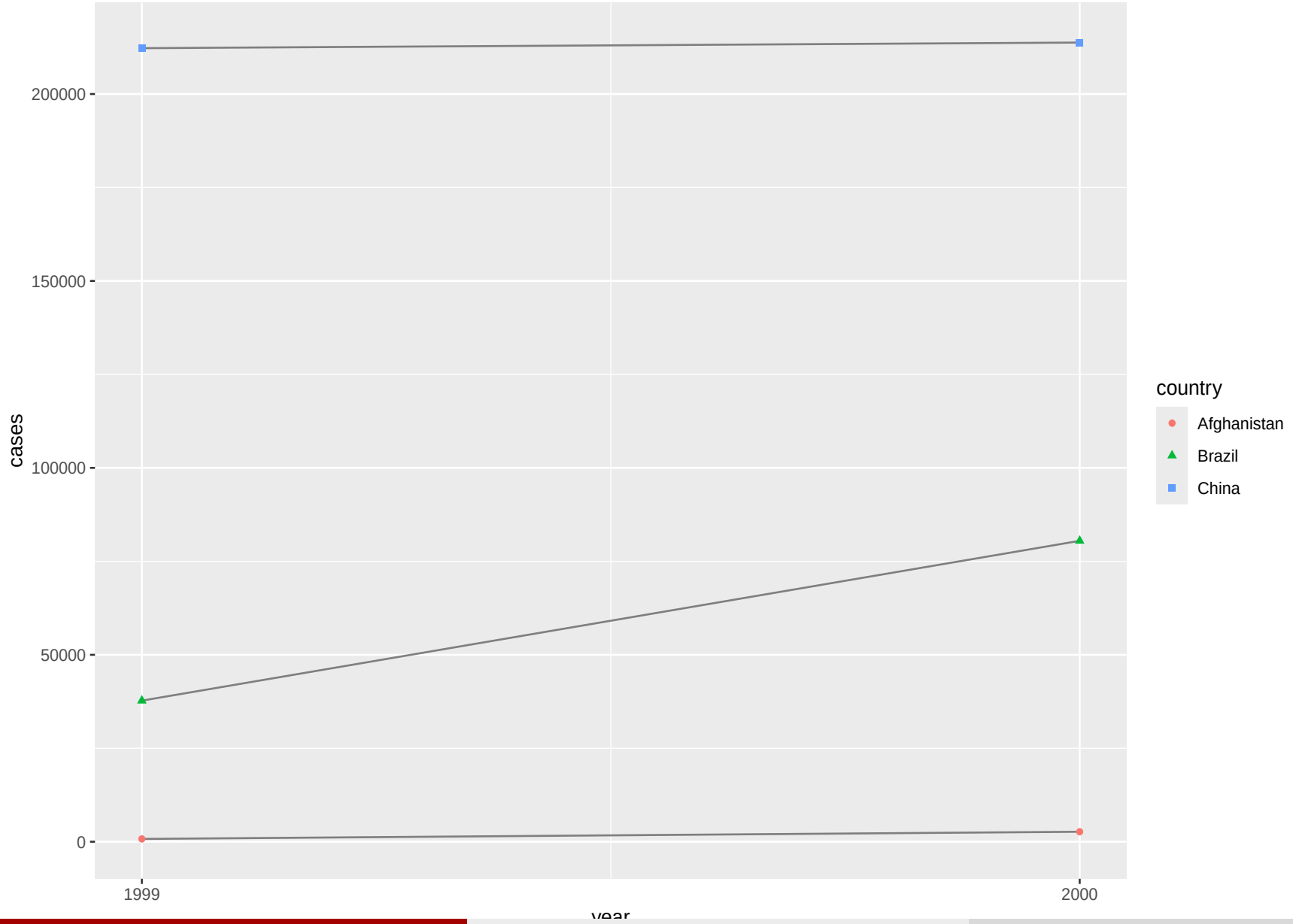
```
table1 |> group_by(year) |>
  summarize(total_cases = sum(cases))
```

```
# A tibble: 2 x 2
   year total_cases
  <dbl>       <dbl>
1  1999      250740
2  2000      296920
```

Visualizar cambios en el tiempo

```
ggplot(table1, aes(x = year, y = cases)) +
geom_line(aes(group = country), color = "grey50") +
geom_point(aes(color = country, shape = country)) +
scale_x_continuous(breaks = c(1999, 2000))
```

Visualizar cambios en el tiempo



Ejercicios

- Para cada una de las tablas anteriores, describa qué representa cada observación y cada columna.
- Describe el proceso que usarías para calcular la tasa de las tablas 2 y 3. Deberás realizar cuatro operaciones:
  1. Extraer el número de casos de tuberculosis por país al año.
  2. Extraer la población coincidente por país al año.
  3. Dividir los casos entre la población y multiplicar por 10000.
  4. Guardar en el lugar correspondiente.

Section 3

Alargamiento de datos: (pivot_longer)

Alargamiento de datos

Los principios de los datos estructurados pueden parecer tan obvios que uno se pregunta si alguna vez encontrará un conjunto de datos que no lo esté. Sin embargo, lamentablemente, la mayoría de los datos reales son no estructurados.

Hay dos razones principales:

1. Los datos suelen organizarse para facilitar algún objetivo distinto al análisis. Por ejemplo, es común estructurarlos para facilitar la entrada de datos, no el análisis.
2. La mayoría de las personas no están familiarizadas con los principios de los datos organizados, y es difícil obtenerlos uno mismo a menos que se dedique mucho tiempo a trabajar con ellos.

Pivoteo de datos

- Esto significa que la mayoría de los análisis reales requieren al menos un poco de ordenamiento.
- Comenzarás por determinar cuáles son las variables y observaciones subyacentes.
- A veces es fácil; otras veces, tendrás que consultar con quienes generaron los datos originalmente.
- A continuación, organizaremos datos para obtener un formato organizado, con las variables en las columnas y las observaciones en las filas.

Pivoteo de datos

- `tidyr` ofrece dos funciones para pivotar datos: `pivot_longer()` y `pivot_wider()`.
- Empezaremos con `pivot_longer()`, ya que es el caso más común

Usaremos el data `billboard` que registra el ranking Billboard de las canciones en el año 2000:

```
# A tibble: 317 x 79
   artist      track date.entered   wk1   wk2   wk3   wk4   wk5
   <chr>       <chr> <date>       <dbl> <dbl> <dbl> <dbl> <dbl>
 1 2 Pac       Baby~ 2000-02-26      87    82    72    77    87
 2 2Ge+her     The ~ 2000-09-02      91    87    92    NA    NA
 3 3 Doors D~  Kryp~ 2000-04-08      81    70    68    67    66
 4 3 Doors D~  Loser 2000-10-21      76    76    72    69    67
 5 504 Boyz    Wobb~ 2000-04-15      57    34    25    17    17
 6 98^0        Give~ 2000-08-19      51    39    34    26    26
 7 A*Teens     Danc~ 2000-07-08      97    97    96    95   100
```

El data billboard

- En este conjunto de datos, cada observación corresponde a una canción.
- Las tres primeras columnas (artista, canción y fecha de entrada) son variables que describen la canción.
- A continuación, tenemos 76 columnas, `wk1` a `wk76`, (semanas 1 a 76) que describen la posición de la canción en cada semana.
- Aquí, los nombres de las columnas representan una variable (la semana) y los valores de las celdas representan otra variable (la posición).
- Para organizar estos datos, usaremos `pivot_longer()`

pivot_longer()

```
billboard |> pivot_longer(
  cols = starts_with("wk"),
  names_to = "week",
  values_to = "rank" )
```

```
# A tibble: 24,092 x 5
   artist track                    date.entered week   rank
   <chr>  <chr>                    <date>       <chr> <dbl>
 1 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk1      87
 2 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk2      82
 3 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk3      72
 4 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk4      77
 5 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk5      87
 6 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk6      94
 7 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk7      99
 8 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk8      NA
 9 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk9      NA
```

Explicación

Después de los datos, hay tres argumentos clave:

- Especifica qué columnas deben *pivotarse* (es decir, qué columnas no son variables).
  - Este argumento usa la misma sintaxis que `select()`, por lo que aquí podríamos usar `!c(artist, track, date.entered)` o `starts_with("wk")`
- `names_to`: especifica el nombre de la columna donde se guardarán los nombres de las semanas
- `values_to`: especifica el nombre de la columna donde se guardarán los valores de las celdas
- Tenga en cuenta que en el código `week` y `rank` están entre comillas porque son variables nuevas que estamos creando; aún no existen en los datos cuando ejecutamos la llamada `pivot_longer()`.

Porqué los NA ?

```
# A tibble: 24,092 x 5
   artist track                    date.entered week   rank
   <chr>  <chr>                    <date>       <chr> <dbl>
 1 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk1      87
 2 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk2      82
 3 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk3      72
 4 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk4      77
 5 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk5      87
 6 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk6      94
 7 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk7      99
 8 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk8      NA
 9 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk9      NA
10 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk10     NA
# i 24,082 more rows
```

Porqué los NA ?

Ahora, centrémonos en el marco de datos resultante.

- ¿Qué ocurre si una canción permanece en el top 100 durante menos de 76 semanas?
- Ejemplo: "Baby Don't Cry" de *2 Pac*. El resultado anterior sugiere que solo estuvo entre las 100 primeras durante 7 semanas
- Las semanas restantes se completan con valores faltantes.
- Estos `NA` en realidad no representan observaciones flatantes; la estructura del conjunto de datos las obligó a existir
- Podemos pedirle a `pivot_longer()` que se deshaga de ellas con la opción `values_drop_na= TRUE`

Uso de values_drop_na= TRUE

```
billboard |> pivot_longer(cols=
                starts_with("wk"),
                names_to = "week",
                values_to = "rank",
                values_drop_na= TRUE )
```

Sin faltantes

```
# A tibble: 5,307 x 5
   artist  track                    date.entered week   rank
   <chr>   <chr>                    <date>       <chr> <dbl>
 1 2 Pac   Baby Don't Cry (Keep... 2000-02-26   wk1      87
 2 2 Pac   Baby Don't Cry (Keep... 2000-02-26   wk2      82
 3 2 Pac   Baby Don't Cry (Keep... 2000-02-26   wk3      72
 4 2 Pac   Baby Don't Cry (Keep... 2000-02-26   wk4      77
 5 2 Pac   Baby Don't Cry (Keep... 2000-02-26   wk5      87
 6 2 Pac   Baby Don't Cry (Keep... 2000-02-26   wk6      94
 7 2 Pac   Baby Don't Cry (Keep... 2000-02-26   wk7      99
 8 2Ge+her The Hardest Part Of ... 2000-09-02   wk1      91
 9 2Ge+her The Hardest Part Of ... 2000-09-02   wk2      87
10 2Ge+her The Hardest Part Of ... 2000-09-02   wk3      92
# i 5,297 more rows
```

Convertir week a numérica

- Estos datos ahora están estructurados (son `tidy`),
- Podríamos hacer que los cálculos futuros sean un poco más fáciles convirtiendo los valores de la semana de cadenas de caracteres a números.
- usando `mutate()` y `parse_number()`
  - `parse_number()` extrae el primer número de una cadena, ignorando todo el resto del texto.

parse_number()

```
billboard_l<- billboard |>
  pivot_longer( cols = starts_with("wk"),
                names_to = "week",
                values_to = "rank",
                values_drop_na =TRUE ) |>
                mutate(week =
                parse_number(week))
billboard_l
```

```
# A tibble: 5,307 x 5
   artist  track                    date.entered  week  rank
   <chr>   <chr>                    <date>       <dbl> <dbl>
 1 2 Pac   Baby Don't Cry (Keep...  2000-02-26       1    87
 2 2 Pac   Baby Don't Cry (Keep...  2000-02-26       2    82
 3 2 Pac   Baby Don't Cry (Keep...  2000-02-26       3    72
 4 2 Pac   Baby Don't Cry (Keep...  2000-02-26       4    77
 5 2 Pac   Baby Don't Cry (Keep...  2000-02-26       5    87
```

Ejercicio

- Use `pivot_longer()` para llevar los datos de la `tabla1` a la forma de la `tabla2`
- Con los datos de acuicultura

```
acuicultura=read.csv2('./data/acuicultura.csv')
head(acuicultura)
```

```
  PISCINA dia OXIGESUP OXIGEFON TEMPSUP TEMPFON SALINSUP SALIN
1       2   1     5.20     4.80   29.67   29.00     7.67     7
2       2   2     4.47     4.13   29.67   29.00     8.00     8
3       2   3     4.53     4.20   30.00   29.00     9.00     9
4       2   4     5.07     4.53   29.67   28.67    10.00    10
5       2   5     4.13     3.87   29.00   28.00    12.00    12
6       2   6     4.40     3.87   28.67   27.67    14.00    14
```

Usar pivot_longer()

- Organizar los datos de tal forma que OXIGESUP OXIGEFON queden en una sola columna

```
# A tibble: 304 x 4
   PISCINA   dia Sitio    Oxigeno
     <int> <int> <chr>      <dbl>
 1       2     1 OXIGESUP    5.2
 2       2     1 OXIGEFON    4.8
 3       2     2 OXIGESUP    4.47
 4       2     2 OXIGEFON    4.13
 5       2     3 OXIGESUP    4.53
 6       2     3 OXIGEFON    4.2
 7       2     4 OXIGESUP    5.07
 8       2     4 OXIGEFON    4.53
 9       2     5 OXIGESUP    4.13
10       2     5 OXIGEFON    3.87
# i 294 more rows
```

Solución

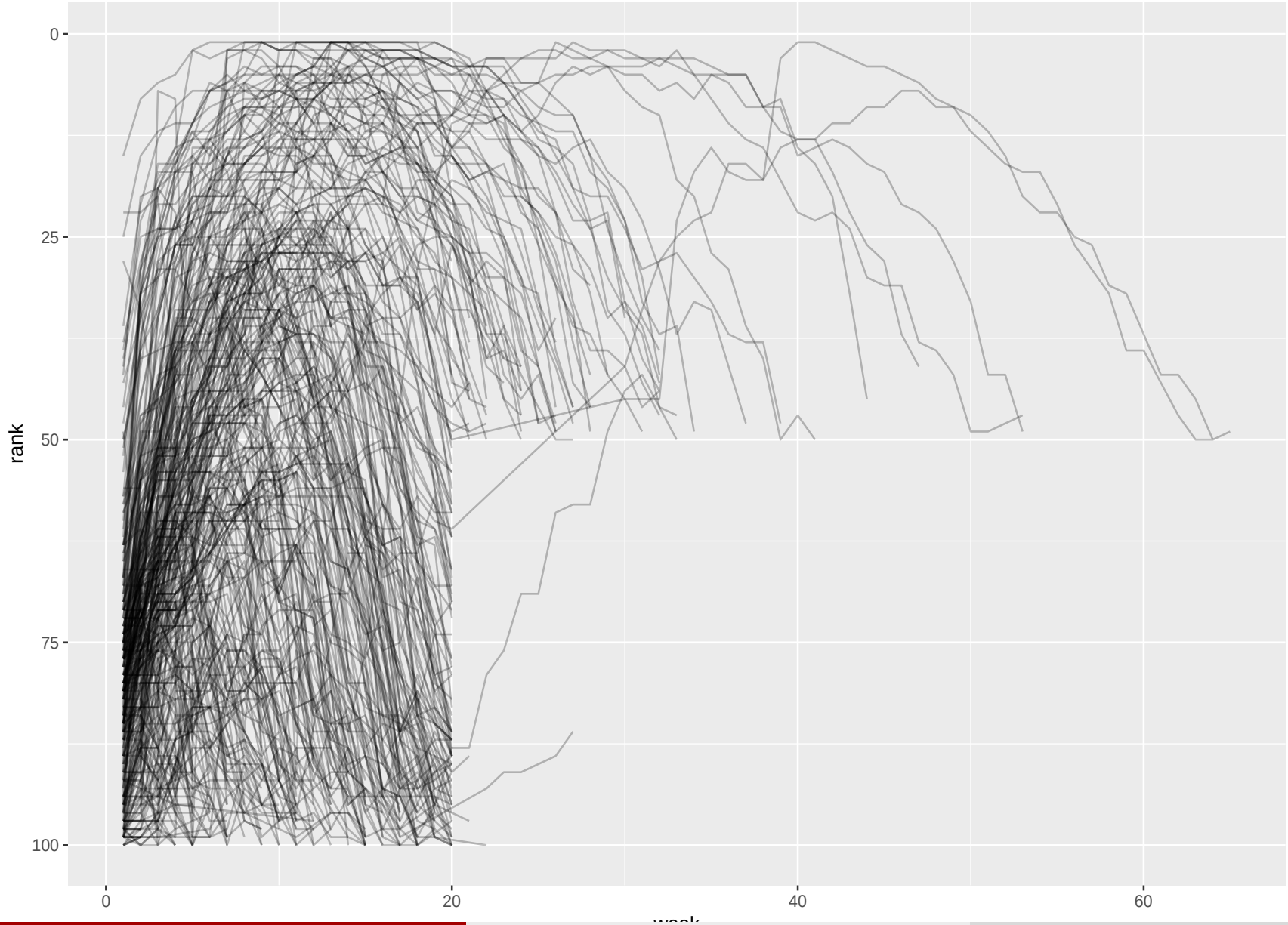
```
oxigeno<-acuicultura |>
  select(PISCINA:OXIGEFON) |> pivot_longer( cols=
        starts_with("OXIGE"), names_to = "Sitio",
        values_to = "Oxigeno" )
oxigeno
```

Visualización de los datos de billboard

- Ahora que tenemos todos los números de semana en una variable y todos los valores de clasificación en otra, podemos visualizar cómo varían las clasificaciones de las canciones con el tiempo.

```
billboard_l |>
ggplot(aes(x = week, y = rank, group = track)) +
geom_line(alpha = 0.25) +
scale_y_reverse()
```

Visualización de los datos de billboard



Visualización de los datos de billboard

Podemos observar que muy pocas canciones permanecen en el top 100 durante más de 20 semanas.

¿Cómo funciona el pivoteo?

- Ahora que hemos visto cómo usar la pivotación para reorganizar los datos, dediquemos un tiempo a comprender mejor su efecto.
- Usaremos un conjunto de datos simple para que sea más fácil ver qué sucede.
- Supongamos que tenemos tres pacientes con identificadores `A`, `B` y `C`, y tomamos dos medidas de presión arterial a cada uno.
- Crearemos el data con `tribble()`, una función práctica para construir pequeños `tibbles` manualmente.

Data de juguete

```
df <- tribble(
~id,~bp1,~bp2,
"A", 100, 120,
"B", 140, 115,
"C", 120, 125
)
df
```

```
# A tibble: 3 x 3
  id      bp1   bp2
  <chr> <dbl> <dbl>
1 A       100   120
2 B       140   115
3 C       120   125
```

Pivotear df

- Queremos que nuestro nuevo conjunto de datos tenga tres variables:
  - `id` (ya existente),
  - `medición` (los nombres de las columnas) y
  - `valor` (los valores de las celdas).
- Para lograrlo, necesitamos pivotar `df` para ponerlo en formato "largo"

Pivotear df

```
df |> pivot_longer(cols = bp1:bp2,
names_to = "measurement",
values_to = "value" )
```

```
# A tibble: 6 x 3
  id    measurement value
  <chr> <chr>       <dbl>
1 A     bp1           100
2 A     bp2           120
3 B     bp1           140
4 B     bp2           115
5 C     bp1           120
6 C     bp2           125
```

Cómo trabaja?

- Los valores de la columna que ya era una variable en el conjunto de datos original (id) deben repetirse una vez por cada columna que se pivotea.

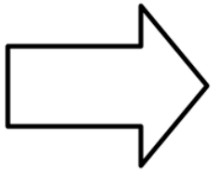
id	bp1	bp2
A	100	120
B	140	115
C	120	125

id	name	value
A	bp1	100
A	bp2	120
B	bp1	140
B	bp2	115
C	bp1	120
C	bp2	125

Cómo trabaja?

- Los nombres de las columnas se convierten en valores de una nueva variable, cuyo nombre se define mediante `names_to`. Deben repetirse una vez para cada fila del conjunto de datos original.

id	bp1	bp2
A	100	120
B	140	115
C	120	125



id	name	value
A	bp1	100
A	bp2	120
B	bp1	140
B	bp2	115
C	bp1	120
C	bp2	125

Cómo trabaja?

- Los valores de celda también se convierten en valores de una nueva variable, cuyo nombre se define como `values_to`. Ellas se desenrollan fila por fila.

id	bp1	bp2
A	100	120
B	140	115
C	120	125

id	name	value
A	bp1	100
A	bp2	120
B	bp1	140
B	bp2	115
C	bp1	120
C	bp2	125

Muchas variables en los nombres de columnas

- Una situación más compleja se presenta cuando se tienen múltiples datos agrupados en los nombres de las columnas y se desea almacenarlos en nuevas variables independientes.
- Por ejemplo, tomemos el conjunto de datos `who2`, la fuente de las tablas 1 2 y 3 anteriores.

```
who2
```

```
# A tibble: 7,240 x 58
   country      year sp_m_014 sp_m_1524 sp_m_2534 sp_m_3544 sp
   <chr>       <dbl>    <dbl>     <dbl>     <dbl>     <dbl>
 1 Afghanistan  1980       NA        NA        NA        NA
 2 Afghanistan  1981       NA        NA        NA        NA
 3 Afghanistan  1982       NA        NA        NA        NA
 4 Afghanistan  1983       NA        NA        NA        NA
 5 Afghanistan  1984       NA        NA        NA        NA
 6 Afghanistan  1985       NA        NA        NA        NA
```

who2

- Este conjunto de datos, recopilado por la Organización Mundial de la Salud, registra información sobre los diagnósticos de tuberculosis.
- Hay dos columnas que ya son variables y son fáciles de interpretar: `country` y `year`.
- Les siguen 56 columnas como `sp_m_014`, `ep_m_4554` y `rel_m_3544`.
- Si miras estas columnas cuidadosamente, notarás que hay un patrón.
- El nombre de cada columna se compone de tres partes separadas por `_`.
  - La primera parte, `sp/rel/ep`, describe el método utilizado para el diagnóstico;
  - la segunda parte, `m/f`, es el género
  - la tercera parte, 014/1524/2534/3544/4554/65, es el rango de edad (014 representa de 0 a 14, por ejemplo).

who2

- Tenemos seis datos registrados en who2:
  - El país y el año (ya en columnas);
  - El método de diagnóstico,
  - La categoría de género y
  - la categoría de rango de edad
  - El número de pacientes en esa categoría (valores de celda).
- Para organizar estos seis datos en seis columnas separadas, usamos `pivot_longer()` con un vector de nombres de columna para `names_to`
- Instrucciones para dividir los nombres de las variables originales en fragmentos para `names_sep`,
- Así como un nombre de columna para `values_to`

who2

```
who2|>pivot_longer(
cols = !(country:year),
names_to = c("diagnosis", "gender", "age"),
names_sep = "_",
values_to = "count"
)
```

```
# A tibble: 405,440 x 6
   country      year diagnosis gender age   count
   <chr>       <dbl> <chr>     <chr>  <chr> <dbl>
 1 Afghanistan  1980 sp        m      014      NA
 2 Afghanistan  1980 sp        m      1524     NA
 3 Afghanistan  1980 sp        m      2534     NA
 4 Afghanistan  1980 sp        m      3544     NA
 5 Afghanistan  1980 sp        m      4554     NA
 6 Afghanistan  1980 sp        m      5564     NA
 7 Afghanistan  1980 sp        m      65       NA
```

Cómo funciona?

Ahora, en lugar de pivotar los nombres de las columnas en una sola, pivotan en varias. Puedes imaginar que esto sucede en dos pasos: primero pivotando y luego separando.

id	x_1	y_2
A	1	2
B	3	4
C	5	6

id	name	number	value
A	x	1	1
A	y	2	2
B	x	1	3
B	y	2	4
C	x	1	5
C	y	2	6

Nombres de datos y variables en los encabezados de columna

El siguiente nivel de complejidad se da cuando los nombres de las columnas incluyen una combinación de valores y nombres de variables. Por ejemplo, tomemos el conjunto de datos `household`

```
household
```

```
# A tibble: 5 x 5
  family dob_child1 dob_child2 name_child1 name_child2
   <int> <date>     <date>     <chr>       <chr>
1      1 1998-11-26 2000-01-29 Susan       Jose
2      2 1996-06-22 NA         Mark        <NA>
3      3 2002-07-11 2004-04-05 Sam         Seth
4      4 2004-10-10 2009-08-27 Craig       Khai
5      5 2000-12-05 2005-02-28 Parker      Gracie
```

household

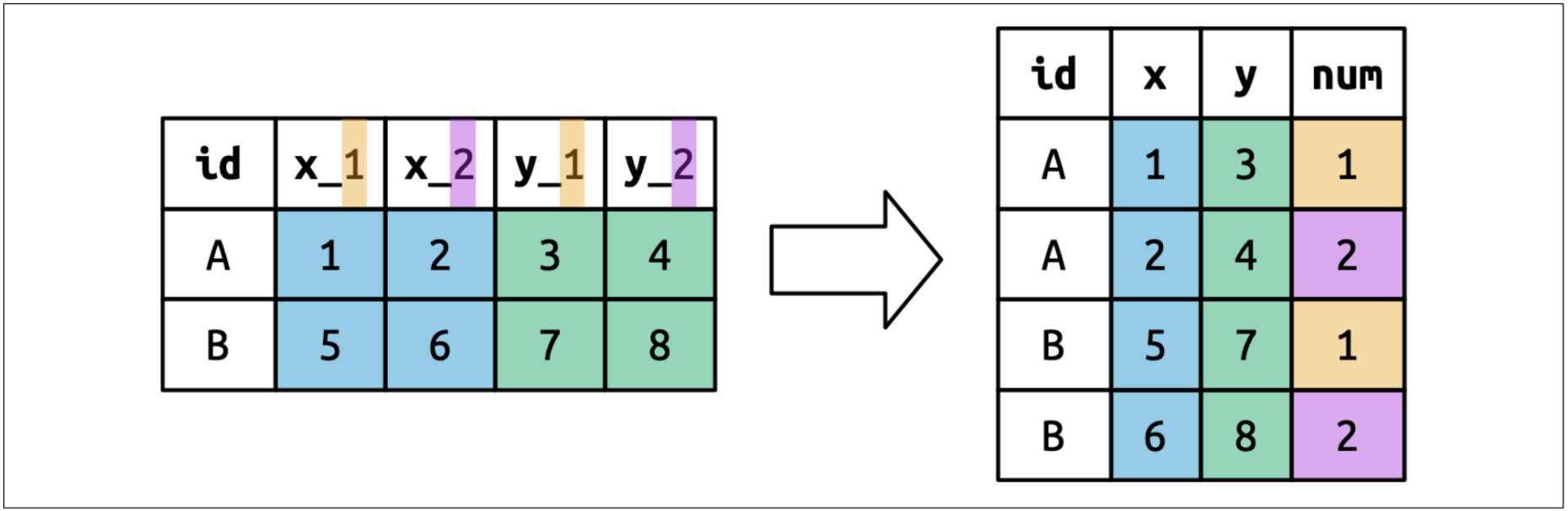
- Este conjunto de datos contiene datos sobre cinco familias, con los nombres y fechas de nacimiento de hasta dos hijos.
- El reto radica en que los nombres de las columnas contienen los nombres de dos variables (fecha de nacimiento, nombre) y los valores de otra (hijo, con valores 1 o 2).
- Para solucionar este problema, necesitamos proporcionar un vector a `names_to`, pero esta vez usamos el centinela especial ".value";
- Este no es el nombre de una variable, sino un valor único que indica a `pivot_longer()` que realice una acción diferente.
- Esto anula el argumento habitual `values_to` para usar el primer componente del nombre de la columna pivotada como nombre de variable en la salida.

Solución

```
household |>
  pivot_longer(
  cols = !family,
  names_to = c(".value", "child"),
  names_sep = "_",
  values_drop_na = TRUE
)
```

```
# A tibble: 9 x 4
  family child  dob        name
   <int> <chr>  <date>     <chr>
1      1 child1 1998-11-26 Susan
2      1 child2 2000-01-29 Jose
3      2 child1 1996-06-22 Mark
4      3 child1 2002-07-11 Sam
5      3 child2 2004-04-05 Seth
6      4 child1 2004-10-10 Craig
```

Esquema



Section 4

Ampliación de datos: pivot_wider()

pivot_wider()

- A continuación, usaremos `pivot_wider()`, que amplía los conjuntos de datos aumentando las columnas y reduciendo las filas, y resulta útil cuando una observación se distribuye en varias filas.
- Esto parece ser menos común en la práctica, pero sí se presenta con frecuencia al trabajar con datos gubernamentales.

pivot_wider()

- A continuación, usaremos `pivot_wider()`, para regresar los datos de oxigeno disuelto en la superficie y fondo a su estructura original, es decir, de

```
# A tibble: 3 x 4
  PISCINA   dia Sitio    Oxigeno
    <int> <int> <chr>      <dbl>
1       2     1 OXIGESUP    5.2
2       2     1 OXIGEFON    4.8
3       2     2 OXIGESUP    4.47
```

a la forma

```
    PISCINA dia OXIGESUP OXIGEFON
1         2   1     5.20     4.80
2         2   2     4.47     4.13
3         2   3     4.53     4.20
4         2   4     5.07     4.53
```

pivot_wider()

```
oxigeno|>pivot_wider(
names_from = Sitio,
values_from = Oxigeno
)
```

```
# A tibble: 152 x 4
   PISCINA   dia OXIGESUP OXIGEFON
     <int> <int>    <dbl>    <dbl>
 1       2     1     5.2      4.8
 2       2     2     4.47     4.13
 3       2     3     4.53     4.2
 4       2     4     5.07     4.53
 5       2     5     4.13     3.87
 6       2     6     4.4      3.87
 7       2     7     4.2      3.93
 8       2     8     3.33     3.13
 9       2     9     3.6      3.27
```

Ejemplo 2

Transformaremos los datos

```
# A tibble: 6 x 3
  id    measurement value
  <chr> <chr>       <dbl>
1 A     bp1           100
2 A     bp2           120
3 B     bp1           140
4 B     bp2           115
5 C     bp1           120
6 C     bp2           125
```

Ejemplo 2

A su versión original

```
df
```

```
# A tibble: 3 x 3
  id      bp1   bp2
  <chr> <dbl> <dbl>
1 A       100   120
2 B       140   115
3 C       120   125
```

Solución

```
df_l|>pivot_wider(
names_from = measurement,
values_from = value
)
```

```
# A tibble: 3 x 3
  id      bp1   bp2
  <chr> <dbl> <dbl>
1 A       100   120
2 B       140   115
3 C       120   125
```

Otro ejemplo

- Comenzaremos analizando `cms_patient_experience`, un conjunto de datos de los Centros de Servicios de `Medicare` y `Medicaid` que recopila datos sobre las experiencias de los pacientes:

cms_patient_experience data

```
cms_patient_experience
```

```
# A tibble: 500 x 5
   org_pac_id org_nm                                measure_cd meas
   <chr>      <chr>                                 <chr>      <chr
 1 0446157747 USC CARE MEDICAL GROUP INC            CAHPS_GRP~ CAHP
 2 0446157747 USC CARE MEDICAL GROUP INC            CAHPS_GRP~ CAHP
 3 0446157747 USC CARE MEDICAL GROUP INC            CAHPS_GRP~ CAHP
 4 0446157747 USC CARE MEDICAL GROUP INC            CAHPS_GRP~ CAHP
 5 0446157747 USC CARE MEDICAL GROUP INC            CAHPS_GRP~ CAHP
 6 0446157747 USC CARE MEDICAL GROUP INC            CAHPS_GRP~ CAHP
 7 0446162697 ASSOCIATION OF UNIVERSITY PHYSI~ CAHPS_GRP~ CAHP
 8 0446162697 ASSOCIATION OF UNIVERSITY PHYSI~ CAHPS_GRP~ CAHP
 9 0446162697 ASSOCIATION OF UNIVERSITY PHYSI~ CAHPS_GRP~ CAHP
10 0446162697 ASSOCIATION OF UNIVERSITY PHYSI~ CAHPS_GRP~ CAHP
# i 490 more rows
```

¿Cómo están organizados los datos?

La unidad central estudiada es una organización, pero cada organización se distribuye en seis filas, con una fila por cada medición realizada en la organización de la encuesta. Podemos ver el conjunto completo de valores para `measure_cd` y `measure_title` usando `distinct()`:

```
cms_patient_experience |>
distinct(measure_cd, measure_title)
```

```
# A tibble: 6 x 2
  measure_cd   measure_title
  <chr>        <chr>
1 CAHPS_GRP_1  CAHPS for MIPS SSM: Getting Timely Care, Appoir
2 CAHPS_GRP_2  CAHPS for MIPS SSM: How Well Providers Communic
3 CAHPS_GRP_3  CAHPS for MIPS SSM: Patient's Rating of Provide
4 CAHPS_GRP_5  CAHPS for MIPS SSM: Health Promotion and Educat
5 CAHPS_GRP_8  CAHPS for MIPS SSM: Courteous and Helpful Offic
6 CAHPS_GRP_12 CAHPS for MIPS SSM: Stewardship of Patient Resc
```

- Ninguna de estas columnas será un nombre de variable especialmente efectivo: `measure_cd` no indica el significado de la variable, y `measure_title` es una oración larga con espacios.
- Por ahora, usaremos `measure_cd` como fuente para los nuevos nombres de columna, pero en un análisis real, podrías crear tus propios nombres de variable, que sean breves y con significado.
- `pivot_wider()` tiene la interfaz opuesta a `pivot_longer()`: en lugar de elegir nuevos nombres de columnas, necesitamos proporcionar las columnas existentes que definen los valores (`values_from`) y el nombre de la columna (`names_from`):

pivot_wider()

```
cms_patient_experience |>
pivot_wider(
names_from = measure_cd,
values_from = prf_rate
)
```

```
# A tibble: 500 x 9
   org_pac_id org_nm           measure_title CAHPS_GRP_1 CAHPS
   <chr>      <chr>            <chr>               <dbl>
 1 0446157747 USC CARE MEDICA~ CAHPS for MI~          63
 2 0446157747 USC CARE MEDICA~ CAHPS for MI~          NA
 3 0446157747 USC CARE MEDICA~ CAHPS for MI~          NA
 4 0446157747 USC CARE MEDICA~ CAHPS for MI~          NA
 5 0446157747 USC CARE MEDICA~ CAHPS for MI~          NA
 6 0446157747 USC CARE MEDICA~ CAHPS for MI~          NA
 7 0446162697 ASSOCIATION OF ~ CAHPS for MI~          59
 8 0446162697 ASSOCIATION OF ~ CAHPS for MI~          NA
```

pivot_wider()

- El resultado no es del todo correcto; parece que aún tenemos varias filas para cada organización.
- Esto se debe a que también necesitamos indicar a `pivot_wider()` qué columna o columnas tienen valores que identifican de forma única cada fila; en este caso, son las variables que empiezan por "org":

```
cms_patient_experience |>
pivot_wider(
id_cols = starts_with("org"),
names_from = measure_cd,
values_from = prf_rate
)
```

pivot_wider()

```
# A tibble: 95 x 8
   org_pac_id org_nm CAHPS_GRP_1 CAHPS_GRP_2 CAHPS_GRP_3 CAHPS
   <chr>      <chr>        <dbl>       <dbl>       <dbl>
 1 0446157747 USC C~          63          87          86
 2 0446162697 ASSOC~          59          85          83
 3 0547164295 BEAVE~          49          NA          75
 4 0749333730 CAPE ~          67          84          85
 5 0840104360 ALLIA~          66          87          87
 6 0840109864 REX H~          73          87          84
 7 0840513552 SCL H~          58          83          76
 8 0941545784 GRITM~          46          86          81
 9 1052612785 COMMU~          65          84          80
10 1254237779 OUR L~          61          NA          NA
# i 85 more rows
# i 1 more variable: CAHPS_GRP_12 <dbl>
```

Cómo trabaja pivot_wider()

- Para entender cómo funciona pivot_wider(), usemos un conjunto de datos simple.
- En esta ocasión tenemos dos pacientes con ID A y B;
- Tenemos tres mediciones de presión arterial en el paciente A y dos en el paciente B:

```
df <- tribble( ~id, ~measurement,~value,
"A", "bp1", 100,
"B", "bp1", 140,
"B", "bp2", 115,
"A", "bp2", 120,
"A", "bp3", 105
)
```

Cómo trabaja pivot_wider()

```
df
```

```
# A tibble: 5 x 3
  id    measurement value
  <chr> <chr>       <dbl>
1 A     bp1           100
2 B     bp1           140
3 B     bp2           115
4 A     bp2           120
5 A     bp3           105
```

Cómo trabaja pivot_wider()

- Tomaremos los valores de la columna `value` y los nombres de la columna `measurement`:

```
df|>pivot_wider(names_from = measurement,
values_from = value
)
```

```
# A tibble: 2 x 4
  id      bp1   bp2   bp3
  <chr> <dbl> <dbl> <dbl>
1 A       100   120   105
2 B       140   115    NA
```

- Para comenzar el proceso, `pivot_wider()` primero debe determinar qué irá en las filas y columnas.

Cómo trabaja pivot_wider()

- Los nuevos nombres de columnas serán los valores únicos de `measurement`:

```
df |>
distinct(measurement) |>
pull()
```

```
[1] "bp1" "bp2" "bp3"
```

- De forma predeterminada, las filas de la salida están determinadas por todas las variables que no entran en los nuevos nombres o valores.
- Estos se llaman `id_cols`. En el ejemplo solo hay una columna, pero en general puede haber cualquier número.

Cómo trabaja pivot_wider()

```
df |>select(-measurement,-value)|> distinct()
```

```
# A tibble: 2 x 1
  id
  <chr>
1 A
2 B
```

Cómo trabaja pivot_wider()

- pivot_wider() luego combina estos resultados para generar un marco de datos vacío:

```
df |>select(-measurement, -value)|>
    distinct()|>
    mutate(x = NA, y = NA, z = NA)
```

```
# A tibble: 2 x 4
  id    x     y     z
  <chr> <lgl> <lgl> <lgl>
1 A     NA    NA    NA
2 B     NA    NA    NA
```

- Luego, completa todos los valores faltantes utilizando los datos de entrada.

Cómo trabaja pivot_wider()

- Qué sucede si hay varias filas en la entrada que corresponden a una celda en la salida.
- El siguiente ejemplo tiene dos filas que corresponden al id A y la medición bp1:

```
df <- tribble(
~id,
~measurement,
~value,
"A", "bp1", 100,
"A", "bp1", 102,
"A", "bp2", 120,
"B", "bp1", 140,
"B", "bp2", 115
)
```

Cómo trabaja pivot_wider()

Si intentamos pivotar este data, obtenemos una salida que contiene columnas de lista. Sobre lo que aprenderemos más adelante.

```
df|>pivot_wider(names_from = measurement,
values_from = value
)
```

```
# A tibble: 2 x 3
  id    bp1       bp2
  <chr> <list>    <list>
1 A     <dbl [2]> <dbl [1]>
2 B     <dbl [1]> <dbl [1]>
```